Canopy Language Roadmap

Bridging the Gap Between Elm's Vision and Modern Development Needs

Canopy Logo

🎯 Executive Vision

Canopy aims to be **the functional language that developers actually want to use**—combining Elm's legendary reliability with the modern features and ecosystem access that today's developers demand.

Core Mission

Transform Canopy from an Elm fork into a **next-generation functional language** that addresses the top community pain points while maintaining the safety and simplicity that makes functional programming delightful.

🔍 Community-Driven Development

This roadmap is built on comprehensive research of the Elm community's most requested features and biggest frustrations:

Top Community Priorities (Addressed by Canopy)

1. 🔧 **Integrated Tooling** - Format, test, review built into compiler (no tool version hell)
2. 🌐 **Enhanced FFI System** - Type-safe Web API access with async/await support
3. ⚡ **Performance & Bundle Optimization** - Built-in runtime, smaller bundles, faster compilation
4. 🏗️ **Essential Abstractions** - Built-in Functor/Applicative patterns (no complex type classes)
5. 🎭 **Rich Error Handling** - Railway-oriented programming and expressive error types
6. 📦 **Evolved Module System** - Improved imports, namespaces, and dependency resolution
7. 🔧 **Advanced Development Features** - Hot reloading, better debugging, enhanced IDE support

---

🗺️ Three-Phase Roadmap Strategy

Phase I: Foundation (6 months)

"*Zero-setup development with integrated tooling*"

Phase II: Innovation (6 months)

"*Essential language features without complexity*"

Phase III: Ecosystem (6 months)

"*Production-ready platform for modern web development*"

🏗️ Phase I: Foundation (Canopy 0.20.0 - 0.22.0)

Theme: "Zero-Setup Development Experience"

Goal: Eliminate setup friction and tool compatibility issues while delivering blazing-fast performance with built-in runtime and integrated development tools.

Milestone 0.20.0: Built-in Runtime & Security (Months 1-2)

🎯 Revolutionary Built-in Runtime

Eliminate elm/core dependency hell

Replace external elm/core dependencies with compiler-generated primitives:

```
-- OLD (elm/core dependency):
import Basics exposing (add, mul)
result = add (mul x 2) y

-- NEW (compiler-generated):
-- No imports needed - arithmetic is built-in
result = (x * 2) + y  -- Direct JavaScript: (x * 2) + y
```

Key Improvements:

- **5-10x faster arithmetic** (direct JS operations vs function calls)
- **50-80% smaller bundles** (no runtime overhead)
- **Zero dependency hell** (core functions built into compiler)
- **Automatic tree-shaking** at primitive level

🛡️ Capability-Based Security Model

World's first secure functional web language

```
-- Each module explicitly declares required capabilities
module WebApp exposing (main)

capabilities
    [ DOM_ACCESS      -- Can manipulate DOM
    , HTTP_CLIENT     -- Can make HTTP requests
    , LOCAL_STORAGE   -- Can access localStorage
    -- NOT: FILE_SYSTEM, NATIVE_CODE, EVAL
    ]

-- Compiler enforces capability restrictions at compile time
main =
    case createDivElement of  -- ✅ Allowed (DOM_ACCESS declared)
        Ok element -> text "Success!"
        Err _ -> text "Error creating element"

-- readFile "config.txt"     -- ❌ Compile error: FILE_SYSTEM not declared
```

Benefits:

- **Prevent supply chain attacks** (explicit capability requirements)
- **Sandbox untrusted code** (principle of least privilege)
- **Enable security audits** (all capabilities visible in source)
- **Runtime enforcement** (impossible to bypass at runtime)

Milestone 0.21.0: Type-Safe FFI Revolution (Months 3-4)

🌟 Type-Safe Web API Access

Finally escape the "hope it works" FFI approach

Method 1: JSDoc-Driven FFI (Recommended for most APIs)

```
// external/dom.js - We control both sides for maximum safety
/**
 * @canopy-type String -> Result DOMError DOMElement
 */
export function createElement(tagName) {
    if (typeof tagName !== 'string') {
        throw new TypeError('createElement: tagName must be a string');
    }
    try {
        return document.createElement(tagName);
    } catch (e) {
        throw new DOMException(`Invalid tag name: ${tagName}`);
    }
}
```

```
-- src/DOM.can - Auto-generated from JSDoc annotations
foreign import javascript "./external/dom.js" exposing (..)

-- Compiler automatically generates:
-- createElement : String -> Result DOMError DOMElement

-- Usage is completely type-safe!
createAppContainer : Html msg
createAppContainer =
    case createElement "div" of
        Ok element ->
            div [] [ text "Container created!" ]
        Err (DOMError.InvalidTag msg) ->
            div [] [ text ("Error: " ++ msg) ]
```

Method 2: TypeScript Declaration Integration (Future milestone)

```
-- Future: Import TypeScript definitions for existing libraries
-- This will be explored in later milestones for maximum ecosystem compatibility
-- Focus in 0.21.0 is on JSDoc-driven approach for better control and safety
```

Method 2: Async/Await Support (Finally!)

```
-- Native async/await syntax for modern JavaScript interop
fetchUserData : Int -> Task HttpError User
fetchUserData userId = async do
    response <- HTTP.get ("/api/users/" ++ String.fromInt userId)
    user <- JSON.decode userDecoder response.body
    return user

-- Compiles to clean Promise-based JavaScript
main =
    Task.attempt HandleUserData (fetchUserData 123)
```

Milestone 0.22.0: Developer Experience Revolution (Months 5-6)

🔧 **Integrated Tooling (No more tool version hell!)**

Built directly into the compiler - zero setup, perfect compatibility

```
# All tools integrated - no separate installations needed
canopy format src/                    # Built-in formatter (elm-format compatible)
canopy format --check src/            # CI-friendly format checking

canopy test                          # Built-in test runner (elm-test compatible)
canopy test --watch                  # Watch mode for TDD
canopy test --filter "User"          # Run specific tests

canopy review                        # Built-in linter (elm-review compatible)
canopy review --fix                  # Auto-fix issues when possible
canopy review --rules minimal        # Use minimal ruleset

# Unified development workflow
canopy dev                          # Combined: format, test, review, compile, serve
# Runs all tools in parallel - instant feedback on every save
```

Smart Formatting

```
-- Before formatting
viewUser user=case user of
  Just u->div[][text u.name,text(String.fromInt u.age)]
  Nothing->div[][text"No user"]

-- After `canopy format` - consistent, readable style
viewUser : Maybe User -> Html msg
viewUser user =
    case user of
        Just u ->
            div []
                [ text u.name
                , text (String.fromInt u.age)
                ]
```

```
        Nothing ->
            div [] [ text "No user" ]
```

Integrated Testing

```
-- Tests run automatically in watch mode
import Test exposing (..)
import Expect

suite : Test
suite =
    describe "User validation"
        [ test "valid email passes" <|
            \_ ->
                validateEmail "user@example.com"
                    |> Expect.equal (Ok (Email "user@example.com"))

        , fuzz string "emails without @ fail" <|
            \email ->
                if not (String.contains "@" email) then
                    validateEmail email
                        |> Expect.err
                else
                    Expect.pass
        ]

-- Output: Live results in terminal + browser test runner
-- ✅ User validation
--   ✅ valid email passes (0.1ms)
--   ✅ emails without @ fail (15.2ms, 100 cases)
```

Smart Code Review

```
-- Built-in linter catches common issues
unnecessaryParens : Int -> Int
unnecessaryParens x = (x + 1)  -- ⚠️ Unnecessary parentheses

-- Auto-fix suggestion:
unnecessaryParens x = x + 1

-- Custom project rules (like elm-review but integrated)
-- canopy.json
{
    "review": {
        "rules": [
            "no-unused-variables",
            "no-debug-log",
            "prefer-map-over-case",
            "enforce-naming-convention"
        ]
```

```
    }
}
```

⚡ **Lightning-Fast Development**

Hot Reloading with State Preservation

```
-- State automatically preserved during hot reloads
type alias Model =
    { count : Int
    , user : Maybe User
    , formData : FormState
    }

-- When you update the view, Model state persists
-- When you update update function, previous actions replay
-- Zero lost state, maximum productivity
```

Enhanced Error Messages

```
-- OLD Elm error:
Type mismatch in `view` function

-- NEW Canopy error:
┌─[src/Main.can:42:12]
│
│   42 │     div [] [ text (String.fromInt user.age) ]
│      │              ── user.age : Maybe Int
│      │                         ── String.fromInt : Int -> String
│
└─ 💡 Hint: Try using `Maybe.map String.fromInt user.age |> Maybe.withDefault "N/A"`

   📖 Learn more: canopy help maybe-mapping
   🔧 Auto-fix: canopy fix src/Main.can:42
```

Incremental Compilation

- **10x faster builds** on typical changes
- **Smart caching** at function and module level
- **Parallel compilation** across CPU cores
- **Watch mode** with instant feedback (format + test + review + compile in <500ms)

🚀 Phase II: Innovation (Canopy 0.23.0 - 0.25.0)

Theme: "Essential Language Features Without Complexity"

Goal: Provide the most requested language features (abstractions, error handling, modules) while maintaining Elm's simplicity and avoiding complexity explosions.

Milestone 0.23.0: Built-in Abstractions (Months 7-8)

🏗️ Essential Built-in Type Classes

Power without complexity - no user-defined type classes

Core Built-in Type Classes (Compiler-provided, not user-definable)

```
-- Built-in Functor support for common types
-- NO user-defined instances - keeps complexity manageable
List.map : (a -> b) -> List a -> List b
Maybe.map : (a -> b) -> Maybe a -> Maybe b
Task.map : (a -> b) -> Task x a -> Task x b
Result.map : (a -> b) -> Result x a -> Result x b

-- Generic map works automatically for these types
map : (a -> b) -> Container a -> Container b
-- where Container is List, Maybe, Task, or Result

-- Usage with unified interface
numbers = [1, 2, 3]
doubled = map (*2) numbers              -- [4, 8, 12]

maybeValue = Just 5
maybeDoubled = map (*2) maybeValue      -- Just 20

taskValue = Task.succeed 10
taskDoubled = map (*2) taskValue        -- Task.succeed 40

resultValue = Ok "hello"
resultMapped = map String.toUpper resultValue  -- Ok "HELLO"
```

Built-in Applicative Pattern

```
-- Built-in support for applicative operations
-- No complex type class syntax - just works
apply : Container (a -> b) -> Container a -> Container b

-- Validation example using built-in applicative
validateUser : String -> String -> Int -> Result (List String) User
validateUser email name age =
    Ok User
        |> apply (validateEmail email)
        |> apply (validateName name)
        |> apply (validateAge age)

-- Multiple Maybe values
combineData : Maybe String -> Maybe Int -> Maybe Bool -> Maybe Record
combineData name age active =
    Ok Record
        |> apply name
        |> apply age
        |> apply active
```

Built-in Do Notation (Syntax sugar only)

```
-- Elegant async/await style - compiles to andThen chains
fetchUserProfile : Int -> Task HttpError UserProfile
fetchUserProfile userId = do
    user <- fetchUser userId
    posts <- fetchUserPosts user.id
    comments <- fetchUserComments user.id
    return { user = user, posts = posts, comments = comments }

-- Desugars to familiar Task.andThen chains:
-- fetchUser userId
--   |> Task.andThen (\user ->
--        fetchUserPosts user.id
--          |> Task.andThen (\posts ->
--               fetchUserComments user.id
--                 |> Task.map (\comments ->
--                      { user = user, posts = posts, comments = comments }))
```

Milestone 0.24.0: Advanced Error Handling (Months 9-10)

🎭 Railway-Oriented Programming

```
-- Union types for comprehensive error modeling
type ValidationError
    = EmailInvalid String
    | PasswordTooShort Int
    | UsernameExists String
    | ServerError String

type alias ValidatedUser =
    { email : Email
    , password : Password
    , username : Username
    }

-- Railway-oriented validation pipeline
validateRegistration : RawForm -> Result (List ValidationError) ValidatedUser
validateRegistration form =
    Ok ValidatedUser
        |> apply (validateEmail form.email)
        |> apply (validatePassword form.password)
        |> apply (validateUsername form.username)

-- Error accumulation instead of fail-fast
validateEmail : String -> Result (List ValidationError) Email
validateEmail email =
    if String.contains "@" email then
        Ok (Email email)
    else
        Err [EmailInvalid "Email must contain @ symbol"]
```

```
-- Pattern matching with error context
handleValidation : Result (List ValidationError) ValidatedUser -> Html msg
handleValidation result =
    case result of
        Ok user ->
            div [] [ text "Registration successful!" ]

        Err errors ->
            div []
                [ h3 [] [ text "Please fix these errors:" ]
                , ul [] (List.map showError errors)
                ]

showError : ValidationError -> Html msg
showError error =
    case error of
        EmailInvalid msg -> li [] [ text ("Email: " ++ msg) ]
        PasswordTooShort min -> li [] [ text ("Password must be at least " ++
String.fromInt min ++ " characters") ]
        UsernameExists name -> li [] [ text ("Username '" ++ name ++ "' is already
taken") ]
        ServerError msg -> li [] [ text ("Server error: " ++ msg) ]
```

Result Combinators

```
-- Powerful combinators for error handling
processUser : Int -> Task AppError ProcessedUser
processUser userId =
    fetchUser userId
        |> Task.recover (handleUserNotFound userId)  -- Fallback on specific errors
        |> Task.retry 3                               -- Auto-retry on network
errors
        |> Task.timeout (30 * 1000)                  -- 30 second timeout
        |> Task.mapError (ServerError << toString)    -- Convert error types
        |> Task.andThen validateUserData              -- Chain dependent operations
```

Milestone 0.25.0: Module System Evolution (Months 11-12)

📦 Namespace System

```
-- Hierarchical module organization
module API.Users.Profile exposing
    ( Profile
    , fetchProfile
    , updateProfile
    )

module API.Users.Settings exposing
    ( UserSettings
    , getSettings
```

```
    , saveSettings
    )

-- Namespace imports prevent name collisions
import API.Users.Profile as Profile
import API.Users.Settings as Settings

-- Clear, unambiguous usage
loadUserData : Int -> Task AppError ( Profile.Profile, Settings.UserSettings )
loadUserData userId =
    Task.map2 Tuple.pair
        (Profile.fetchProfile userId)
        (Settings.getSettings userId)
```

Re-export System

```
-- Create clean public APIs by re-exporting
module API exposing
    ( module Users
    , module Posts
    , module Comments
    )

-- Re-export everything from sub-modules
import API.Users as Users
import API.Posts as Posts
import API.Comments as Comments

-- Usage
import API

user = API.Users.fetchProfile 123
posts = API.Posts.getUserPosts user.id
```

Package-Level Configuration

```
// canopy.json - Enhanced dependency resolution
{
    "type": "application",
    "source-directories": ["src"],
    "dependencies": {
        "direct": {
            "canopy/core": "1.0.0",
            "canopy/html": "1.0.0",
            "canopy/http": "1.0.0"
        },
        "indirect": {}
    },
    "namespace": "com.mycompany.myapp",
    "capabilities": [
        "DOM_ACCESS",
```

```
        "HTTP_CLIENT",
        "LOCAL_STORAGE"
    ]
}
```

🌐 Phase III: Ecosystem (Canopy 0.26.0 - 0.28.0)

Theme: "Production-Ready Platform"

Goal: Complete the platform with advanced performance features, ecosystem integration, and professional-grade development tools that make Canopy the clear choice for production applications.

Milestone 0.26.0: Advanced Development Tools (Months 13-14)

🔧 Integrated Development Environment

Language Server Protocol (LSP) Integration

```
-- Real-time type information and suggestions
type alias User = { name : String, age : Int }

users : List User
users =
    [ { name = "Alice", age = 30 }
    , { name = "Bob", age =  }  -- ← IDE shows: "Expected Int"
    --                    ^ Auto-complete suggests: 25, 30, 35...
    ]

-- Intelligent refactoring
-- Right-click on 'User' type → "Extract to module" → "Rename all occurrences"
```

Advanced Debugging

```
-- Time-travel debugging with full state inspection
debug : String -> a -> a  -- Automatically removed in production builds
debug label value =
    -- Browser dev tools show:
    -- [Timeline] Step 23: "user-validation" = { email: "alice@example.com", ... }
    -- [State Tree] Model changes over time
    -- [Action Log] All Msg values with timestamps
    Native.Debug.log label value

-- Breakpoint debugging
debugger : a -> a  -- Pauses execution in dev tools
debugger value = Native.Debug.breakpoint value
```

Performance Profiling

```
-- Built-in performance monitoring
benchmark : String -> (() -> a) -> a
```

```
benchmark name thunk =
    -- Automatic performance tracking:
    -- - Function execution time
    -- - Memory allocation
    -- - Render performance
    -- - Bundle size impact
    Native.Benchmark.measure name thunk

slowFunction : List Int -> Int
slowFunction numbers =
    benchmark "sum-calculation" <|
        \() -> List.foldl (+) 0 numbers
```

Milestone 0.27.0: WebAssembly Integration (Months 15-16)

⚡ WebAssembly First-Class Support

```
-- Direct WebAssembly imports with type safety
foreign import wasm "./crypto.wasm" as Crypto

-- Automatically generated bindings from WASM interface
sha256 : String -> Bytes
sha256 = Crypto.sha256Hash

aesEncrypt : String -> String -> Bytes
aesEncrypt = Crypto.aesGcmEncrypt

-- Usage with performance guarantees
hashPassword : String -> String
hashPassword password =
    password
        |> String.toUtf8
        |> sha256
        |> Bytes.toHex
        -- Runs at native WASM speed with memory safety
```

WASM Module Integration

```
-- Import existing WASM modules with type safety
foreign import wasm "./image-processing.wasm" as ImageLib

-- Type-safe bindings automatically generated
resizeImage : Int -> Int -> ImageData -> Task WasmError ImageData
resizeImage width height imageData =
    ImageLib.resize width height imageData

-- Compose WASM operations
processImage : ImageData -> Task WasmError ImageData
processImage image =
    image
        |> resizeImage 800 600
```

```
        |> Task.andThen (ImageLib.sharpen 1.5)
        |> Task.andThen ImageLib.optimizeColors
```

WebAssembly Compilation Target

```
-- Compile Canopy functions to WASM for maximum performance
{-# COMPILE_TO_WASM #-}
intensiveCalculation : List Float -> Float
intensiveCalculation numbers =
    numbers
        |> List.map (\x -> x * x + sin x)
        |> List.foldl (+) 0
        |> sqrt

-- Compiler automatically:
-- 1. Compiles function to WASM
-- 2. Generates JavaScript wrapper
-- 3. Handles memory management
-- 4. Provides type-safe interface
```

GPU Computing via WebGPU

```
-- WebGPU compute shaders as first-class citizens
compute shader ParticleUpdate : ComputeShader
    { particles : Buffer Particle
    , deltaTime : Float
    , gravity : Vec3
    } ->
    { updatedParticles : Buffer Particle }

updateParticles : List Particle -> Float -> Task GPUError (List Particle)
updateParticles particles dt =
    WebGPU.dispatch ParticleUpdate
        { particles = Buffer.fromList particles
        , deltaTime = dt
        , gravity = vec3 0 -9.8 0
        }
        |> Task.map (.updatedParticles >> Buffer.toList)
```

Milestone 0.28.0: Complete Platform (Months 17-18)

🏗️ Full-Stack Canopy Applications

Server-Side Rendering

```
-- Universal Canopy applications
module Pages.UserProfile exposing (page)

-- Runs on both client and server
page : UserId -> ServerContext -> Task PageError (Html Msg)
page userId context = async do
```

```
    user <- API.fetchUser userId
    posts <- API.fetchUserPosts user.id
    return (viewUserProfile user posts)

-- Server renders initial HTML + hydrates on client
-- Same code, different execution environments
```

Static Site Generation

```
-- Compile-time page generation
static pages : List (Path, Html Never)
pages =
    [ ("/", homePage)
    , ("/about", aboutPage)
    , ("/contact", contactPage)
    ]

-- Generates optimized static HTML at build time
-- Zero JavaScript for content pages
-- Progressive enhancement for interactive features
```

Comprehensive Package Ecosystem (56 Essential Packages)

```
# Core Language Foundation (8 packages)
canopy install canopy/core         # Built-in runtime with 10x performance
canopy install canopy/maybe        # Safe null handling with comprehensive utilities
canopy install canopy/result       # Railway-oriented programming with error
accumulation
canopy install canopy/list         # Performance-optimized immutable lists
canopy install canopy/dict         # Efficient dictionaries with advanced operations
canopy install canopy/set          # Mathematical set operations
canopy install canopy/array        # High-performance arrays for large datasets
canopy install canopy/tuple        # Tuple utilities and pairing operations

# Web Platform & Comprehensive APIs (35 packages)
# Core Web Platform
canopy install canopy/html         # Type-safe HTML with accessibility features
canopy install canopy/svg          # Scalable vector graphics with mathematical
precision
canopy install canopy/css          # Type-safe CSS with modern layout systems
canopy install canopy/dom          # Capability-based DOM access
canopy install canopy/events       # Advanced gesture recognition and input handling
canopy install canopy/browser      # Browser integration with storage APIs
canopy install canopy/platform     # Device detection and progressive enhancement

# Modern Web APIs & Standards
canopy install canopy/webgpu       # GPU computing and high-performance graphics
canopy install canopy/webassembly  # WebAssembly integration with memory safety
canopy install canopy/service-worker # PWA functionality and offline support
canopy install canopy/web-components # Custom elements with shadow DOM
canopy install canopy/intersection-observer # Efficient lazy loading
```

```
canopy install canopy/resize-observer # Container-based responsive design
canopy install canopy/payment-request # Secure payment processing
canopy install canopy/pwa          # Complete Progressive Web App suite

# Device & Hardware APIs
canopy install canopy/geolocation  # GPS and location services with privacy controls
canopy install canopy/sensors      # Device sensors and hardware integration
canopy install canopy/camera       # Camera and video capture with MediaStreams
canopy install canopy/bluetooth    # Web Bluetooth API for device connectivity
canopy install canopy/gamepad      # Game controller support for interactive
applications

# User Interface & System APIs
canopy install canopy/speech       # Speech recognition and synthesis for voice
interfaces
canopy install canopy/clipboard    # Clipboard operations for copy/paste
functionality
canopy install canopy/notifications # Push notifications and local notification
system
canopy install canopy/share        # Web Share API for native sharing integration
canopy install canopy/fullscreen   # Fullscreen API for immersive experiences
canopy install canopy/wakeLock     # Screen Wake Lock API to prevent device sleep

# Storage & Background APIs
canopy install canopy/filesystem   # File System Access API for local file
operations
canopy install canopy/indexeddb    # Client-side database for complex data storage
canopy install canopy/backgroundFetch # Background Fetch API for large downloads

# NEW: Advanced Media & Communication APIs (6 packages)
canopy install canopy/video        # Comprehensive video processing and streaming
API
canopy install canopy/audio        # Professional audio processing and synthesis API
canopy install canopy/webrtc       # Real-time communication with peer-to-peer
connectivity
canopy install canopy/screenCapture # Screen recording and live streaming
capabilities
canopy install canopy/webstreams   # Streaming data processing and transformation
canopy install canopy/encoding     # Text encoding, decoding, and data format
conversion

# Data & Communication (6 packages)
canopy install canopy/json         # JSON with automatic derivation and validation
canopy install canopy/http         # HTTP client with retry, caching, and middleware
canopy install canopy/graphql      # GraphQL client with query generation
canopy install canopy/websockets   # Real-time communication with auto-reconnection
canopy install canopy/sse          # Server-sent events with multiplexing
canopy install canopy/protobuf     # Protocol buffer serialization

# Security & Performance (9 packages)
canopy install canopy/capability   # Capability-based security system
canopy install canopy/crypto       # Web Crypto API with capability protection
```

```
canopy install canopy/csp          # Content Security Policy management
canopy install canopy/permissions  # Permission management with graceful degradation
canopy install canopy/lazy         # Lazy evaluation and deferred computation
canopy install canopy/virtual-dom  # Efficient virtual DOM with smart diffing
canopy install canopy/memoization  # Function caching for expensive computations
canopy install canopy/web-workers  # Background processing and parallelism
canopy install canopy/streaming    # Memory-efficient large dataset processing

# Developer Experience (1 package)
# Note: Core dev tools (test, debug, hot-reload, benchmark) built into compiler
canopy install canopy/devtools     # Browser developer tools integration

# Graphics & Media (4 packages)
canopy install canopy/animation    # Declarative animations with timeline control
canopy install canopy/canvas       # HTML5 Canvas with functional drawing primitives
canopy install canopy/webgl        # 3D graphics and shader programming
canopy install canopy/media        # Audio/video processing with Web Audio API

# Full-Stack Deployment (3 packages)
canopy install canopy/ssr          # Server-side rendering and static generation
canopy install canopy/deploy       # Production deployment with optimization
```

Ecosystem Highlights:

- 🚀 **56 packages** with comprehensive Web API coverage for modern development
- 🔒 **Security-first** with capability-based access control throughout
- ⚡ **Performance-optimized** with built-in runtime and GPU acceleration
- 🛠️ **Zero-setup development** with testing, debugging, and hot-reload built into compiler
- 🌐 **Complete Web API support** covering device sensors, media capture, real-time communication, and more
- 📱 **Native app-like experiences** with PWA, screen capture, wake lock, and background processing
- 🎮 **Rich interactions** supporting gamepad, speech, clipboard, and advanced media capabilities
- 🎬 **Professional Media APIs** with video streaming, audio synthesis, WebRTC, and screen recording

See packages.md for detailed specifications of all 56 packages

Advanced Build System

```
// canopy.json - Production configuration
{
    "optimization": {
        "bundleSplitting": true,        // Code splitting by route
        "treeshaking": "aggressive",    // Dead code elimination
        "compression": "brotli",        // Maximum compression
        "inlining": "smart",           // Inline small functions
        "minification": "advanced"      // Aggressive minification
    },
    "targets": {
        "browsers": "> 1%, not dead",   // Browser compatibility
```

```
        "node": "18",                    // Server-side target
        "webassembly": true             // Enable WASM optimizations
    },
    "performance": {
        "budgets": {
            "initial": "150kb",          // Initial bundle size limit
            "chunks": "50kb"            // Chunk size limit
        }
    }
}
```

📈 Expected Impact & Metrics

Performance Improvements

- **10x faster compilation** (incremental builds, parallel processing, integrated tooling)
- **5-10x smaller bundles** (built-in runtime, dead code elimination)
- **2-5x faster runtime** (direct JavaScript operations, WASM integration)
- **Zero tool setup time** (format, test, review built-in)

Developer Experience

- **Integrated tooling** (no version mismatches, single command workflow)
- **Zero elm/core dependency issues** (built-in runtime)
- **Type-safe ecosystem access** (simplified FFI system)
- **Instant feedback** (format + test + review + compile in <500ms)

Language Capabilities

- **Essential abstractions** (built-in Functor/Applicative patterns)
- **Railway-oriented programming** (comprehensive error handling)
- **Modern async programming** (do notation, async/await)
- **Full-stack applications** (SSR, static generation)

Ecosystem Growth

- **Package compatibility** (TypeScript, WebAssembly integration)
- **Community engagement** (RFC process, contributor onboarding)
- **Industry adoption** (production-ready features, performance)
- **Educational impact** (functional programming accessibility)

🛣️ Migration & Compatibility Strategy

Seamless Elm Compatibility

```
# Existing Elm projects work unchanged
elm make src/Main.elm            # Old
canopy make src/Main.elm         # New - identical behavior

# But with integrated tooling benefits
canopy dev                       # Format + test + review + compile + serve
# vs old workflow:
```

```
# elm-format --yes src/ && elm-test && elm-review && elm make src/Main.elm

# Gradual migration path
canopy migrate-to-builtin ./     # Remove elm/core dependency
canopy add-capabilities ./       # Add capability declarations
canopy modernize-ffi ./          # Convert to type-safe FFI
```

Version Migration Guide

- **0.19.x → 0.20.x**: Automatic built-in runtime (no code changes)
- **0.20.x → 0.22.x**: Integrated tooling benefits (replace separate tools with `canopy dev`)
- **0.22.x → 0.25.x**: Gradual essential abstractions adoption
- **0.25.x → 0.28.x**: Enhanced features (backwards compatible)

🎯 Success Metrics

Community Adoption

- **GitHub Stars**: Target 10,000+ (currently ~500 for elm-lang/elm)
- **Package Downloads**: 50,000+ monthly by 0.25.0
- **Production Users**: 100+ companies using Canopy by 0.28.0
- **Developer Satisfaction**: >90% would recommend (vs ~60% for current Elm)

Technical Excellence

- **Build Speed**: <5 seconds for medium projects (vs 30+ seconds typical)
- **Bundle Size**: <150KB initial load (vs 300-500KB typical React apps)
- **Runtime Performance**: Top 10% on JS framework benchmarks
- **Type Safety**: Zero production runtime type errors

Ecosystem Health

- **Core Packages**: 50+ official packages covering major use cases
- **Community Packages**: 500+ community packages
- **Contributors**: 200+ contributors across core and ecosystem
- **Documentation**: Complete docs with tutorials and examples

---

📞 Call to Action

For the Community

Elm Developers: Help us prioritize features by joining our RFC discussions. Your experience with Elm's limitations directly informs our development priorities.

JavaScript Developers: Try Canopy's type-safe FFI system. Show us which JavaScript libraries need first-class integration.

Functional Programming Enthusiasts: Contribute to our higher-kinded types implementation. Help us design the type class system that the community actually wants.

Performance Engineers: Benchmark Canopy against alternatives. Help us optimize the compiler and runtime for real-world applications.

How to Contribute

1. 🐛 **Issues**: Report bugs, request features, share use cases
2. 💡 **RFCs**: Propose language features through our RFC process
3. 🔧 **Code**: Contribute to compiler, tooling, or ecosystem packages
4. 📚 **Documentation**: Write tutorials, guides, and examples
5. 💬 **Community**: Help others in discussions and Stack Overflow

Funding & Support

Canopy development is accelerated by:

- **Individual Sponsors**: Support core development through GitHub Sponsors
- **Corporate Partners**: Companies using Canopy in production
- **Grant Funding**: Applied for Mozilla Open Source Support, others
- **Volunteer Contributors**: The backbone of our community

🚀 The Future is Functional

Canopy represents more than just another programming language—it's a **vision of functional programming without the setup hassle**.

By combining Elm's legendary reliability with integrated tooling, essential language features, and zero-configuration development, Canopy eliminates every friction point that has limited functional programming adoption.

This is functional programming that just works, out of the box.

The question isn't whether functional programming will become mainstream—it's whether we'll build the language that makes it effortless to adopt.

Join us in building that future.

Last updated: January 2025 Next roadmap review: July 2025

Contributing: See CONTRIBUTING.md for development setup **Community**: Join our Discord and forum **Sponsor**: Support development on GitHub Sponsors